

Customer Churn Analysis

End-to-End Data & Machine Learning Project

Shehraz Sarwar

Aspiring Data Scientist | IBM Certified Data Analyst

Ex Data Science Intern @10Pearls | Section Leader Stanford CIP '25

 [linkedin.com/in/shehraz-sarwar-ghouri-321394247](https://www.linkedin.com/in/shehraz-sarwar-ghouri-321394247)

 github.com/ShehrazSarwar/Customer-Churn-Analysis-Project

SQL Server · Python / XGBoost · Power BI

Streamlit · scikit-learn · Jupyter

May 2025

Contents

1	Project Overview	3
1.1	Project Objectives	3
1.2	Domain Applicability	4
2	Tech Stack	4
3	Project Folder Structure	4
4	Dataset Description	5
4.1	Feature Categories	5
4.2	Customer Status Breakdown	6
5	ETL Pipeline — SQL Server	6
5.1	Script 1 — Database Creation & Data Ingestion	6
5.2	Script 2 — KPI Analysis Queries	8
5.3	Script 3 — Data Quality Check	9
5.4	Script 4 — Null Imputation Strategy	11
5.5	Script 5 — Production Table Creation	11
5.6	Script 6 — Analytical Views	12
6	Power BI Dashboard	13
6.1	Page 1 — Summary Dashboard	14
6.1.1	Top-Line KPIs	14
6.1.2	Demographic Insights	14
6.1.3	Account Behaviour Insights	14
6.1.4	Service Insights	15
6.1.5	Geographic Insights	15
6.1.6	Churn Category Breakdown	16
6.2	Page 2 — Prediction Dashboard	16
6.2.1	Predicted Churner Profile	17
7	Machine Learning Pipeline	17
7.1	Step 1 — Data Extraction from SQL Server	17
7.2	Step 2 — Exploratory Data Analysis	18
7.2.1	Outlier Detection — Boxplots	18
7.2.2	Correlation Heatmap	20
7.3	Step 3 — Data Preprocessing	21

7.4	Step 4 — Model Training & Comparison	23
7.4.1	Model Comparison Results	24
7.5	Step 5 — Model Selection Rationale	24
7.6	Step 6 — Model Serialisation	25
7.7	Step 7 — Prediction on New Joiners	25
8	ChurnRadar — Streamlit Dashboard	26
8.1	How It Works	27
8.2	Dashboard Tabs	27
8.3	Risk Classification	28
9	Key Findings & Business Insights	28
9.1	Top Churn Risk Factors	28
9.2	Retention Strategy Recommendations	29
10	How to Run This Project	29
10.1	Prerequisites	29
10.2	Step-by-Step Setup	30
11	Conclusion	30

1 Project Overview

Customer churn — the point at which a customer decides to stop using a product or service — is one of the most expensive problems any subscription or service-driven business can face. The cost of acquiring a new customer typically far exceeds the cost of keeping an existing one, which makes early identification of at-risk customers not just useful but genuinely strategic.

This project builds a complete, production-oriented churn analysis and prediction system from the ground up. It covers every layer of a modern data pipeline: raw data ingestion and transformation in SQL Server, exploratory analysis and business intelligence reporting in Power BI, machine learning model development in Python, and finally a live interactive prediction dashboard built with Streamlit. While the dataset originates from a telecommunications context, the entire architecture — the ETL schema, the modelling choices, the dashboard design — is deliberately domain-agnostic and applies equally well to banking, e-commerce, SaaS platforms, healthcare subscriptions, or any field where retaining existing customers is a business priority.

At a Glance			
6,418	1,732	27.0%	388
Total Customers	Total Churners	Churn Rate	Predicted At-Risk

1.1 Project Objectives

- Build a structured, reproducible ETL pipeline to ingest and clean raw customer data using T-SQL on SQL Server.
- Conduct thorough KPI analysis and exploratory reporting through an interactive two-page Power BI dashboard.
- Train, evaluate, and compare multiple machine learning classifiers to predict customer churn accurately.
- Deploy an interactive real-time prediction dashboard (ChurnRadar) using Streamlit for non-technical business users.
- Derive actionable retention strategies grounded in both historical analysis and forward-looking model predictions.

1.2 Domain Applicability

Although the dataset originates from a telecom provider, the methodology transfers directly to any subscription or service business. The churn signals explored here have equivalent analogues across sectors:

Domain	Churn Signal Equivalent
Banking	Account inactivity, fee disputes, competitor switching
SaaS	Feature underutilisation, rising support ticket volume
E-commerce	Declining purchase frequency, cart abandonment
Healthcare	Missed appointments, insurance plan switching
Media / Streaming	Days-since-last-login, drop in content engagement

2 Tech Stack

Layer	Tools & Technologies
Data Storage	Microsoft SQL Server — T-SQL, BULK INSERT, Views
Business Intelligence	Power BI Desktop — DAX, slicers, KPI cards, charts
Programming Language	Python 3.12 (Jupyter Notebook + Streamlit)
ML Libraries	scikit-learn, XGBoost, LightGBM, joblib
Data Handling	Pandas, NumPy
Visualisation	Matplotlib, Seaborn
Live Dashboard	Streamlit (ChurnRadar application)
Model Persistence	joblib serialisation (.joblib files)
DB Connector	SQLAlchemy + pyodbc (ODBC Driver 17 for SQL Server)

3 Project Folder Structure

The actual repository folder structure is shown below, matching the project as organised on disk:

Listing 1: *Actual project folder structure*

```
Customer Churn Analysis Project/
|
|-- Data & Resources/           # Raw CSV data and supporting files
```

```

|
|-- Machine Learning Predictions/
|   |-- Churn_Predictions.ipynb           # Full ML pipeline notebook
|   |-- churn_preprocessor.joblib         # Fitted preprocessing pipeline
|   |-- baseline_xgb_model.joblib        # Trained XGBoost model
|   |-- high_risk_churn_list.csv          # Exported predictions for Power
    BI
|   |-- new_joiners_data.csv              # New joiner records
|
|-- Power BI Dashboard Screenshots/      # PNG exports of both report pages
|
|-- SQL/                                  # All T-SQL scripts
|   |-- 01_etl_pipeline.sql               # DB creation, staging, BULK
    INSERT
|   |-- 02_kpi_analysis.sql               # KPI queries (gender, state,
    revenue)
|   |-- 03_data_quality_check.sql         # Null audit across all columns
|   |-- 04_imputation_strategy.sql        # Null fill rules
|   |-- 05_clean_table.sql                # prod_Churn with ISNULL()
|   |-- 06_analytical_views.sql           # vw_ChurnData & vw_JoinData
|
|-- Streamlit App/
|   |-- churn_prediction.py                # ChurnRadar dashboard
|
|-- Churn Analysis.pbix                   # Power BI report file
|-- Power BI Dashboard.pdf               # Exported PDF of dashboard
|-- Power BI Report - Insights.pdf        # Written insights document

```

4 Dataset Description

The dataset contains **6,418 customer records** across **32 attributes**, spanning demographic details, subscribed services, account financials, and churn classification labels. Data was initially ingested as a flat CSV file into a SQL Server staging table (`dbo.stg_Churn`).

4.1 Feature Categories

- **Demographics:** Customer_ID, Gender, Age, Married, State, Number_of_Referrals
- **Account Info:** Tenure_in_Months, Value_Deal, Contract, Paperless_Billing, Payment_Method
- **Services:** Phone_Service, Multiple_Lines, Internet_Service, Internet_Type, Online_Security, Online_Backup, Device_Protection_Plan, Premium_Support, Stream-

ing_TV, Streaming_Movies, Streaming_Music, Unlimited_Data

- **Financials:** Monthly_Charge, Total_Charges, Total_Refunds, Total_Extra_Data_Charges, Total_Long_Distance_Charges, Total_Revenue
- **Labels:** Customer_Status (Churned / Stayed / Joined), Churn_Category, Churn_Reason

4.2 Customer Status Breakdown

Status	Description	Role in Project
Churned	Customers who have left	Positive class in ML training
Stayed	Active, retained customers	Negative class in ML training
Joined	Newly acquired customers	Prediction target (no label yet)

5 ETL Pipeline — SQL Server

The data engineering layer was built entirely in Microsoft SQL Server using T-SQL. The pipeline follows a two-stage architecture: a raw staging table for initial data load, followed by a cleaned production table for all downstream analysis and reporting. All six SQL scripts are presented in full below.

5.1 Script 1 — Database Creation & Data Ingestion

This script creates the `db_Churn` database from scratch, defines the staging table schema with all 32 columns and appropriate data types, and bulk-loads the raw CSV using `BULK INSERT`. Any pre-existing database instance is dropped cleanly first, making the pipeline fully reproducible.

Listing 2: *ETL Pipeline: Database creation and CSV ingestion*

```

1  /**
2  -- ETL Pipeline: Customer Churn Data Ingestion
3  -- Creates database, staging table, and loads raw CSV data into SQL
   Server
4  -- for further cleaning, transformation, and Power BI analysis. */
5
6  USE master;
7  GO
8
9  -- Drop database if it already exists
10 IF EXISTS (SELECT 1 FROM sys.databases WHERE name = 'db_Churn')
11 BEGIN

```

```
12  ALTER DATABASE db_Churn
13  SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
14  DROP DATABASE db_Churn;
15  END;
16  GO
17
18  -- Create and select the database
19  CREATE DATABASE db_Churn;
20  GO
21  USE db_Churn;
22  GO
23
24  -- Create staging table
25  CREATE TABLE dbo.stg_Churn (
26      Customer_ID          VARCHAR(50),
27      Gender                VARCHAR(20),
28      Age                  INT,
29      Married              VARCHAR(10),
30      State                VARCHAR(100),
31      Number_of_Referrals  INT,
32      Tenure_in_Months    INT,
33      Value_Deal           VARCHAR(100),
34      Phone_Service        VARCHAR(10),
35      Multiple_Lines       VARCHAR(20),
36      Internet_Service     VARCHAR(10),
37      Internet_Type        VARCHAR(50),
38      Online_Security      VARCHAR(10),
39      Online_Backup        VARCHAR(10),
40      Device_Protection_Plan VARCHAR(10),
41      Premium_Support      VARCHAR(10),
42      Streaming_TV         VARCHAR(10),
43      Streaming_Movies     VARCHAR(10),
44      Streaming_Music      VARCHAR(10),
45      Unlimited_Data       VARCHAR(10),
46      Contract            VARCHAR(50),
47      Paperless_Billing    VARCHAR(10),
48      Payment_Method       VARCHAR(50),
49      Monthly_Charge       DECIMAL(10,2),
50      Total_Charges        DECIMAL(12,2),
51      Total_Refunds        DECIMAL(12,2),
52      Total_Extra_Data_Charges DECIMAL(12,2),
53      Total_Long_Distance_Charges DECIMAL(12,2),
54      Total_Revenue        DECIMAL(12,2),
55      Customer_Status      VARCHAR(50),
56      Churn_Category       VARCHAR(100),
```



```

57      Churn_Reason          VARCHAR(255)
58  );
59  GO
60
61  -- Load CSV data into staging table
62  BULK INSERT dbo.stg_Churn
63  FROM 'C:\...\Customer_Data.csv'      -- Update path to match your
        directory
64  WITH (
65      FIRSTROW = 2,
66      FIELDTERMINATOR = ',',
67      TABLOCK
68  );
69  GO

```

5.2 Script 2 — KPI Analysis Queries

Six analytical queries were written to explore key business metrics before building the Power BI report. These cover gender distribution, contract-type segmentation, multi-dimensional gender-by-contract analysis, state-level geography, revenue by customer status, and internet type classification.

Listing 3: *KPI analysis queries*

```

1  USE db_Churn;
2  GO
3
4  -- 1. Gender-wise Customer Distribution
5  SELECT
6      Gender,
7      COUNT(*) AS TotalByGender,
8      SUM(COUNT(*)) OVER() AS TotalCustomers,
9      CAST(100.0 * COUNT(*) / SUM(COUNT(*)) OVER()
10          AS DECIMAL(5,2)) AS Percentage
11  FROM dbo.stg_Churn
12  GROUP BY Gender;
13
14  -- 2. Contract-wise Customer Distribution
15  SELECT
16      Contract,
17      COUNT(*) AS TotalbyCategory,
18      SUM(COUNT(*)) OVER() AS Total,
19      CAST(100.0 * COUNT(*) / SUM(COUNT(*)) OVER()
20          AS DECIMAL(5,2)) AS Percentage
21  FROM dbo.stg_Churn

```

```

22 GROUP BY Contract;
23
24 -- 3. Gender + Contract Segmentation
25 SELECT
26     Gender,
27     Contract,
28     COUNT(*) AS TotalbyCategory,
29     CAST(100.0 * COUNT(*) / SUM(COUNT(*)) OVER()
30         AS DECIMAL(5,2)) AS Percentage
31 FROM dbo.stg_Churn
32 GROUP BY Contract, Gender;
33
34 -- 4. State-wise Customer Distribution
35 SELECT
36     State,
37     COUNT(*) AS TotalbyCategory,
38     CAST(100.0 * COUNT(*) / SUM(COUNT(*)) OVER()
39         AS DECIMAL(5,2)) AS Percentage
40 FROM dbo.stg_Churn
41 GROUP BY State
42 ORDER BY TotalbyCategory DESC;
43
44 -- 5. Revenue by Customer Status
45 SELECT
46     Customer_Status,
47     COUNT(*) AS TotalCount,
48     SUM(Total_Revenue) AS TotalRevByStatus,
49     CAST(
50         100.0 * SUM(Total_Revenue) /
51         (SELECT SUM(Total_Revenue) FROM dbo.stg_Churn)
52     AS DECIMAL(5,2)) AS Percentage
53 FROM dbo.stg_Churn
54 GROUP BY Customer_Status
55 ORDER BY TotalRevByStatus;
56
57 -- 6. Distinct Internet Types
58 SELECT DISTINCT Internet_Type
59 FROM dbo.stg_Churn;

```

5.3 Script 3 — Data Quality Check

A comprehensive null-count audit across all 32 columns was run in a single query using the `COUNT(*) - COUNT(column)` pattern. This delivers an instant null-profile of the entire dataset without requiring column-by-column inspection.

Listing 4: *Null audit across all 32 columns*

```

1  USE db_Churn;
2  GO
3
4  SELECT
5      COUNT(*) - COUNT(Customer_ID)      AS Customer_ID_Null_Count ,
6      COUNT(*) - COUNT(Gender)           AS Gender_Null_Count ,
7      COUNT(*) - COUNT(Age)              AS Age_Null_Count ,
8      COUNT(*) - COUNT(Married)          AS Married_Null_Count ,
9      COUNT(*) - COUNT(State)            AS State_Null_Count ,
10     COUNT(*) - COUNT(Number_of_Referrals)
11                                     AS Number_of_Referrals_Null_Count ,
12     COUNT(*) - COUNT(Tenure_in_Months)
13                                     AS Tenure_in_Months_Null_Count ,
14     COUNT(*) - COUNT(Value_Deal)        AS Value_Deal_Null_Count ,
15     COUNT(*) - COUNT(Phone_Service)     AS Phone_Service_Null_Count ,
16     COUNT(*) - COUNT(Multiple_Lines)    AS Multiple_Lines_Null_Count ,
17     COUNT(*) - COUNT(Internet_Service)
18                                     AS Internet_Service_Null_Count ,
19     COUNT(*) - COUNT(Internet_Type)     AS Internet_Type_Null_Count ,
20     COUNT(*) - COUNT(Online_Security)
21                                     AS Online_Security_Null_Count ,
22     COUNT(*) - COUNT(Online_Backup)     AS Online_Backup_Null_Count ,
23     COUNT(*) - COUNT(Device_Protection_Plan)
24                                     AS
25                                     Device_Protection_Plan_Null_Count
26                                     ,
27     COUNT(*) - COUNT(Premium_Support)
28                                     AS Premium_Support_Null_Count ,
29     COUNT(*) - COUNT(Streaming_TV)      AS Streaming_TV_Null_Count ,
30     COUNT(*) - COUNT(Streaming_Movies)
31                                     AS Streaming_Movies_Null_Count ,
32     COUNT(*) - COUNT(Streaming_Music)
33                                     AS Streaming_Music_Null_Count ,
34     COUNT(*) - COUNT(Unlimited_Data)     AS Unlimited_Data_Null_Count ,
35     COUNT(*) - COUNT(Contract)          AS Contract_Null_Count ,
36     COUNT(*) - COUNT(Paperless_Billing)
37                                     AS Paperless_Billing_Null_Count ,
38     COUNT(*) - COUNT(Payment_Method)    AS Payment_Method_Null_Count ,
39     COUNT(*) - COUNT(Monthly_Charge)    AS Monthly_Charge_Null_Count ,
40     COUNT(*) - COUNT(Total_Charges)     AS Total_Charges_Null_Count ,
41     COUNT(*) - COUNT(Total_Refunds)     AS Total_Refunds_Null_Count ,
42     COUNT(*) - COUNT(Total_Extra_Data_Charges)
43                                     AS Total_Extra_Data_Null_Count ,
44     COUNT(*) - COUNT(Total_Long_Distance_Charges)

```

```

43                                     AS Total_Long_Distance_Null_Count ,
44 COUNT(*) - COUNT(Total_Revenue) AS Total_Revenue_Null_Count ,
45 COUNT(*) - COUNT(Customer_Status)
46                                     AS Customer_Status_Null_Count ,
47 COUNT(*) - COUNT(Churn_Category) AS Churn_Category_Null_Count ,
48 COUNT(*) - COUNT(Churn_Reason)   AS Churn_Reason_Null_Count
49 FROM dbo.stg_Churn;

```

5.4 Script 4 — Null Imputation Strategy

Rather than dropping records with missing values, a domain-aware imputation strategy was defined before any transformation was applied. The core logic: a `NULL` in a service column simply means the customer does not subscribe to that service — it is not missing data but rather absent data. Churn classification nulls are grouped into *Others* to preserve all records without inventing meaning.

Column	Fill Value & Rationale
Value_Deal	None — customer has no deal
Multiple_Lines	No — single-line or no phone service
Internet_Type	None — no internet service subscribed
Online_Security	No — service not subscribed
Online_Backup	No — service not subscribed
Device_Protection_Plan	No — service not subscribed
Premium_Support	No — service not subscribed
Streaming_TV	No — service not subscribed
Streaming_Movies	No — service not subscribed
Streaming_Music	No — service not subscribed
Unlimited_Data	No — service not subscribed
Churn_Category	Others — no explicit reason recorded
Churn_Reason	Others — no explicit reason recorded

5.5 Script 5 — Production Table Creation

The cleaned, imputed dataset is written into `dbo.prod_Churn` via a `SELECT INTO` statement. All `ISNULL()` transformations are applied inline — no intermediate temporary tables, no multi-step process.

Listing 5: *Production table with inline null imputation*

```

1 SELECT
2     Customer_ID ,

```

```

3      Gender ,
4      Age ,
5      Married ,
6      State ,
7      Number_of_Referrals ,
8      Tenure_in_Months ,
9      ISNULL(Value_Deal ,      'None') AS Value_Deal ,
10     Phone_Service ,
11     ISNULL(Multiple_Lines ,      'No') AS Multiple_Lines ,
12     Internet_Service ,
13     ISNULL(Internet_Type ,      'None') AS Internet_Type ,
14     ISNULL(Online_Security ,      'No') AS Online_Security ,
15     ISNULL(Online_Backup ,      'No') AS Online_Backup ,
16     ISNULL(Device_Protection_Plan , 'No') AS Device_Protection_Plan ,
17     ISNULL(Premium_Support ,      'No') AS Premium_Support ,
18     ISNULL(Streaming_TV ,      'No') AS Streaming_TV ,
19     ISNULL(Streaming_Movies ,      'No') AS Streaming_Movies ,
20     ISNULL(Streaming_Music ,      'No') AS Streaming_Music ,
21     ISNULL(Unlimited_Data ,      'No') AS Unlimited_Data ,
22     Contract ,
23     Paperless_Billing ,
24     Payment_Method ,
25     Monthly_Charge ,
26     Total_Charges ,
27     Total_Refunds ,
28     Total_Extra_Data_Charges ,
29     Total_Long_Distance_Charges ,
30     Total_Revenue ,
31     Customer_Status ,
32     ISNULL(Churn_Category , 'Others') AS Churn_Category ,
33     ISNULL(Churn_Reason ,      'Others') AS Churn_Reason
34
35 INTO dbo.prod_Churn
36 FROM dbo.stg_Churn;

```

5.6 Script 6 — Analytical Views

Two SQL views were created on top of the production table to logically separate historical churn data from newly joined customers. This separation keeps the ML training data clean and ensures the prediction pipeline always operates on the correct subset.

Listing 6: *Analytical views for BI and ML consumption*

```

1 USE db_Churn;
2 GO

```

```
3
4  -- View 1: Historical data for churn analysis and Power BI reporting
5  IF OBJECT_ID('dbo.vw_ChurnData', 'V') IS NOT NULL
6      DROP VIEW dbo.vw_ChurnData;
7  GO
8
9  CREATE VIEW dbo.vw_ChurnData AS
10 SELECT *
11 FROM dbo.prod_Churn
12 WHERE Customer_Status IN ('Churned', 'Stayed');
13 GO
14
15 -- View 2: New joiners for churn prediction
16 IF OBJECT_ID('dbo.vw_JoinData', 'V') IS NOT NULL
17     DROP VIEW dbo.vw_JoinData;
18 GO
19
20 CREATE VIEW dbo.vw_JoinData AS
21 SELECT *
22 FROM dbo.prod_Churn
23 WHERE Customer_Status = 'Joined';
24 GO
```

6 Power BI Dashboard

A two-page interactive Power BI report was built on top of the SQL views, with slicers for Monthly Charge Range and Marital Status enabling dynamic filtering across all visuals. Both pages are shown in full below.

6.1 Page 1 — Summary Dashboard



Figure 1: Power BI Summary Dashboard — KPIs, demographic analysis, service usage, geographic churn, and churn category distribution

6.1.1 Top-Line KPIs



6.1.2 Demographic Insights

Gender: Female customers account for around 64.15% of all churners (1,111 out of 1,732), making them the dominant churn segment despite forming roughly half the customer base. Male customers contributed the remaining 35.85% (621 churners).

Age: Churn rate climbs steadily with age. The above-50 age group carries the highest individual churn rate at 31.0% and also the highest total customer count (2,800+), making them the most critical segment by volume. Customers in the 35–50 range follow at 24.0%, while the youngest cohort (under 20) shows only marginal churn.

6.1.3 Account Behaviour Insights

Contract Type: Month-to-Month customers churn at 46.5% — more than four times the rate of One Year (11.0%) and seventeen times the rate of Two Year (2.7%) contract

holders. Contract length is among the clearest structural predictors of retention in this dataset.

Payment Method: Customers paying by Mailed Check churn at 37.8%, Bank Withdrawal at 34.4%, and Credit Card at just 14.8%. The pattern suggests digital-first payment methods correlate with higher account engagement and lower churn risk.

Tenure: Customers with tenure above 24 months contribute the highest absolute churn volume (2,100+). While counterintuitive, this reflects the size of that cohort — they are also the largest segment — and may signal unresolved long-term dissatisfaction.

6.1.4 Service Insights

Internet Type: Fiber Optic customers churn at 57.9% — by far the highest of any internet tier. DSL (32.4%) and Cable (36.0%) are considerably lower. Customers with no internet service have the lowest churn at 16.8%.

Service Subscription Table (Churn By Services):

Service	Not Subscribed (churn %)	Subscribed (churn %)
Online_Security	84.6	15.4
Premium_Support	83.5	16.5
Online_Backup	71.9	28.1
Device_Protection_Plan	71.0	29.0
Streaming_Movies	56.0	44.0
Multiple_Lines	54.8	45.2
Paperless_Billing	25.4	74.6
Phone_Service	9.4	90.6
Internet_Service	6.3	93.7

The data reveals that customers who do *not* subscribe to security and support add-ons churn at much higher rates — a strong argument for service bundling as a retention tool.

6.1.5 Geographic Insights

Top 5 states by churn rate: Jammu & Kashmir (57.2%), Assam (38.1%), Jharkhand (34.5%), Chhattisgarh (30.5%), Delhi (29.9%). These regions should be the primary focus of regional retention campaigns.

6.1.6 Churn Category Breakdown

Churn Category	Customer Count
Competitor	761
Attitude	301
Dissatisfaction	300
Price	196
Other	174

Competitor attraction is the dominant churn driver, representing nearly 44% of all churn cases, followed by attitude issues and general dissatisfaction.

6.2 Page 2 — Prediction Dashboard

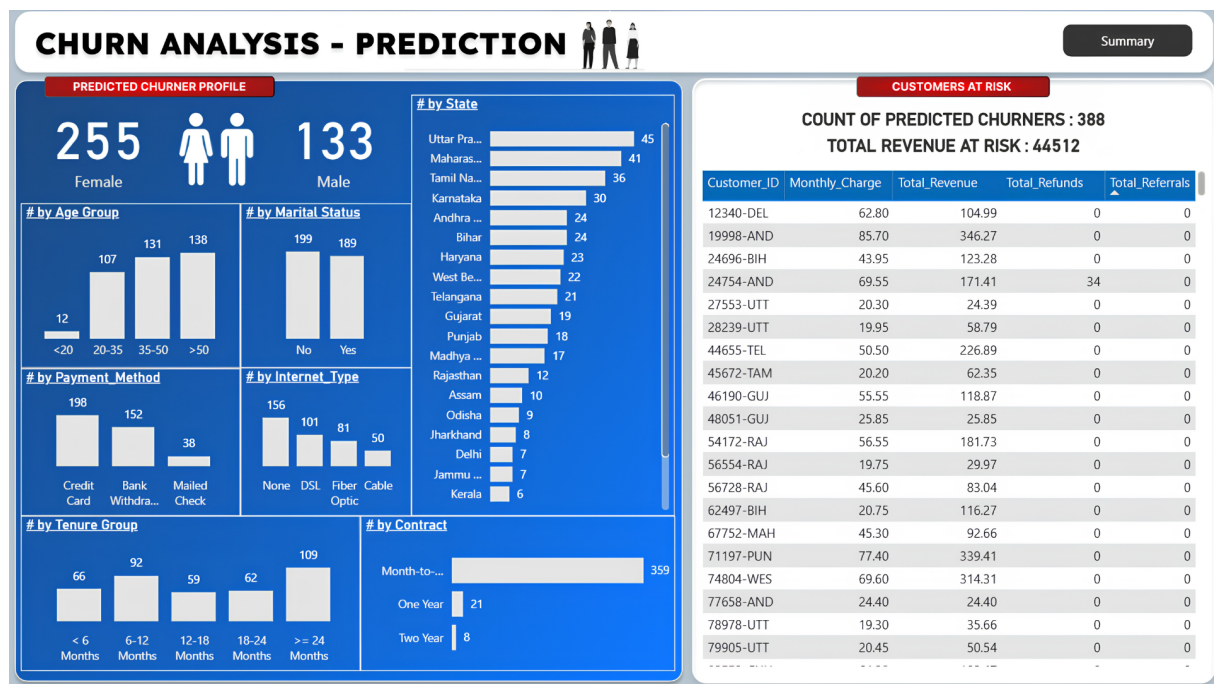


Figure 2: Power BI Prediction Dashboard — XGBoost-predicted churning profiles and at-risk customer detail table

The second page integrates the XGBoost model's exported predictions directly into Power BI, allowing business stakeholders to view and act on churn risk without touching any Python code.

388	44,512
Predicted Churners	Total Revenue at Risk

6.2.1 Predicted Churner Profile

- **Gender:** 255 female, 133 male — mirrors the historical pattern.
- **Age:** Above-50 (138) and 35–50 (131) groups dominate; under-20 barely registers (12).
- **Marital Status:** Non-married (199) slightly outnumber married (189).
- **Contract:** 359 of 388 predicted churners are on Month-to-Month contracts; 21 on One Year; 8 on Two Year.
- **Tenure:** Above-24 months (109) leads, followed by 6–12 months (92) and 35–50 months (138 customers in adjacent age bin).
- **Payment Method:** Credit Card (198), Bank Withdrawal (152), Mailed Check (38).
- **Internet Type:** No service (156), DSL (101), Fiber Optic (81), Cable (50).
- **Geography:** Uttar Pradesh (45), Maharashtra (41), Tamil Nadu (36), Karnataka (30).

The right panel of the dashboard provides a scrollable, exportable table of individual at-risk customers including `Customer_ID`, `Monthly_Charge`, `Total_Revenue`, `Total_Refunds`, and `Total_Referrals` — giving the retention team a directly actionable contact list.

7 Machine Learning Pipeline

The ML component was developed in a Jupyter Notebook (`Churn_Predictions.ipynb`) using Python 3.12. Data is fetched directly from SQL Server via SQLAlchemy, ensuring the notebook always uses the latest cleaned production data without any manual CSV handling.

7.1 Step 1 — Data Extraction from SQL Server

Listing 7: *SQLAlchemy connection and data extraction*

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4 import numpy as np
5 from sqlalchemy import create_engine, text
6
7 # Establish database connection using Windows Authentication
```

```

8 engine = create_engine(
9     "mssql+pyodbc://localhost\\SQLEXPRESS/db_Churn"
10    "?driver=ODBC+Driver+17+for+SQL+Server"
11    "&trusted_connection=yes"
12 )
13 conn = engine.connect()
14
15 # Verify available tables
16 tables = pd.read_sql(
17     "SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES", conn
18 )
19 print(tables)
20 # Output:
21 #      TABLE_NAME
22 # 0      stg_Churn
23 # 1     prod_Churn
24 # 2  vw_ChurnData
25 # 3  vw_JoinData
26
27 # Load historical data (for training) and new joiners (for prediction)
28 df_churn = pd.read_sql("SELECT * FROM vw_ChurnData", conn)
29 df_join  = pd.read_sql("SELECT * FROM vw_JoinData", conn)
30
31 # Export new joiners to CSV for later reference
32 df_join.to_csv("new_joiners_data.csv", index=False)

```

Two DataFrames are created: `df_churn` (5,007 historical records with churn labels, used for training and testing) and `df_join` (411 new joiners with no churn label, used as the prediction target).

7.2 Step 2 — Exploratory Data Analysis

Before any modelling work, the numerical features were inspected for outliers and inter-feature correlations. This step informed both the decision to keep extreme values and the final feature set going into the preprocessing pipeline.

7.2.1 Outlier Detection — Boxplots

Boxplots were generated across all nine numerical columns to assess the spread and flag any values that might distort model training.

Listing 8: *Boxplot grid across all numerical features*

```

1 # Select all numerical columns

```

```

2 num_cols = df_churn.select_dtypes(include='number').columns
3 n_cols   = len(num_cols)
4
5 # Define a 3-column grid
6 cols = 3
7 rows = int(np.ceil(n_cols / cols)) # 9 / 3 = 3 rows
8
9 plt.figure(figsize=(cols * 5, rows * 4))
10 for i, col in enumerate(num_cols):
11     plt.subplot(rows, cols, i + 1)
12     sns.boxplot(y=df_churn[col])
13     plt.title(col)
14
15 plt.tight_layout()
16 plt.show()

```

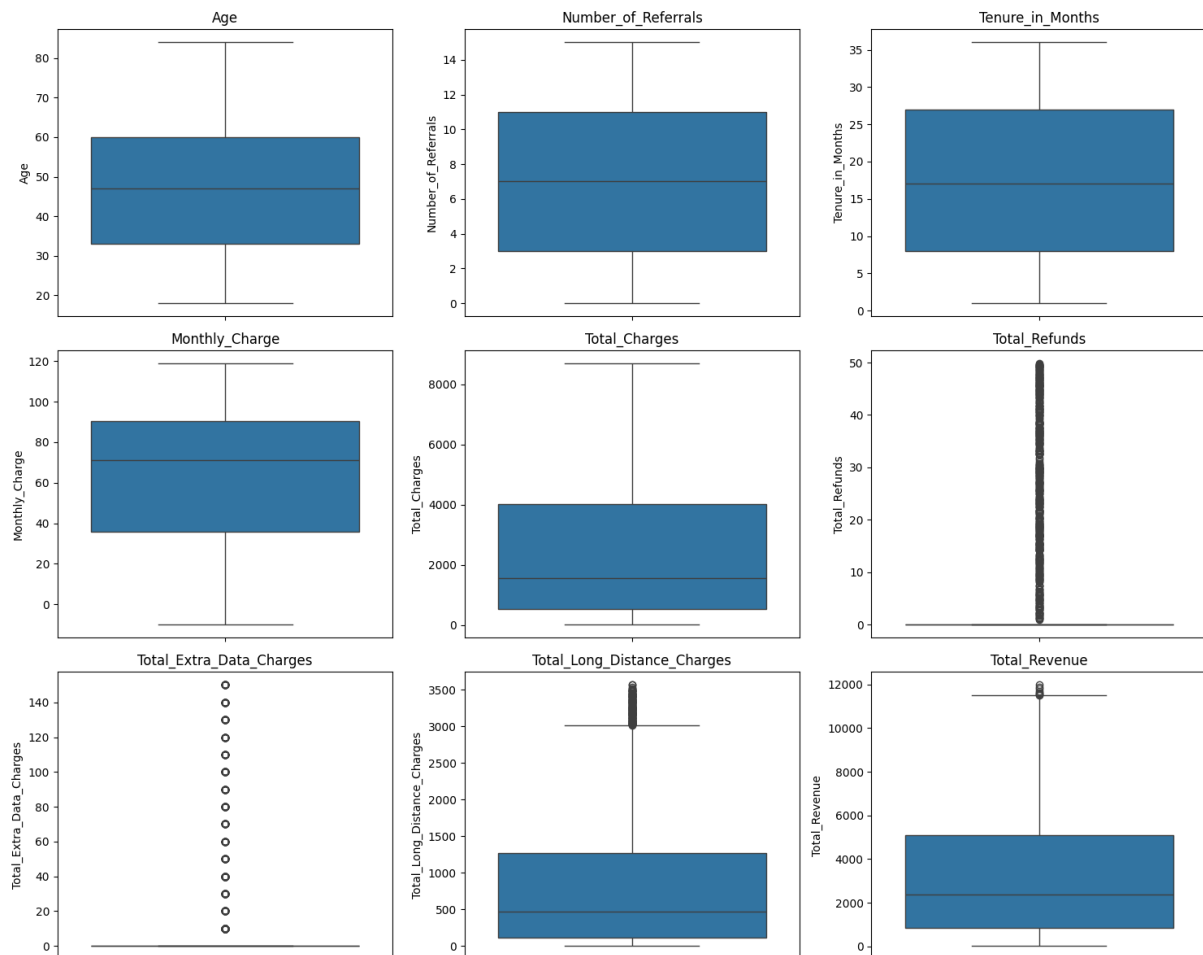


Figure 3: Boxplot grid for all nine numerical features — outlier distribution across Age, Referrals, Tenure, Charges, and Revenue columns

Several columns show visible outliers above the upper whisker. `Total_Extra_Data_Charges` has a cluster of values up to 150, `Total_Long_Distance_Charges` extends past 3,000, and `Total_Refunds` shows a dense band of outliers reaching up to 50. Despite these extreme values, no records were removed. The reasoning is deliberate — these are not data entry errors or measurement noise; they represent genuine high-value customers with unusual billing profiles. Dropping them would cause the model to underperform on exactly the kind of high-revenue customer the business most wants to retain.

7.2.2 Correlation Heatmap

A Pearson correlation matrix was computed across all numerical features plus the binary-encoded `Customer_Status` target variable to understand linear relationships before encoding.

Listing 9: *Correlation heatmap of numerical features against Customer Status*

```
1 # Encode target as binary and append to numerical columns
2 corr = pd.concat(
3     [
4         df_churn[num_cols],
5         df_churn["Customer_Status"].map({"Stayed": 0, "Churned": 1}),
6     ],
7     axis=1,
8 ).corr()
9
10 plt.figure(figsize=(11, 6))
11 sns.heatmap(
12     data=corr,
13     annot=True,
14     linewidths=0.5,
15     cmap='coolwarm'
16 )
17 plt.title("Correlation Heatmap: Numerical Values")
18 plt.show()
```

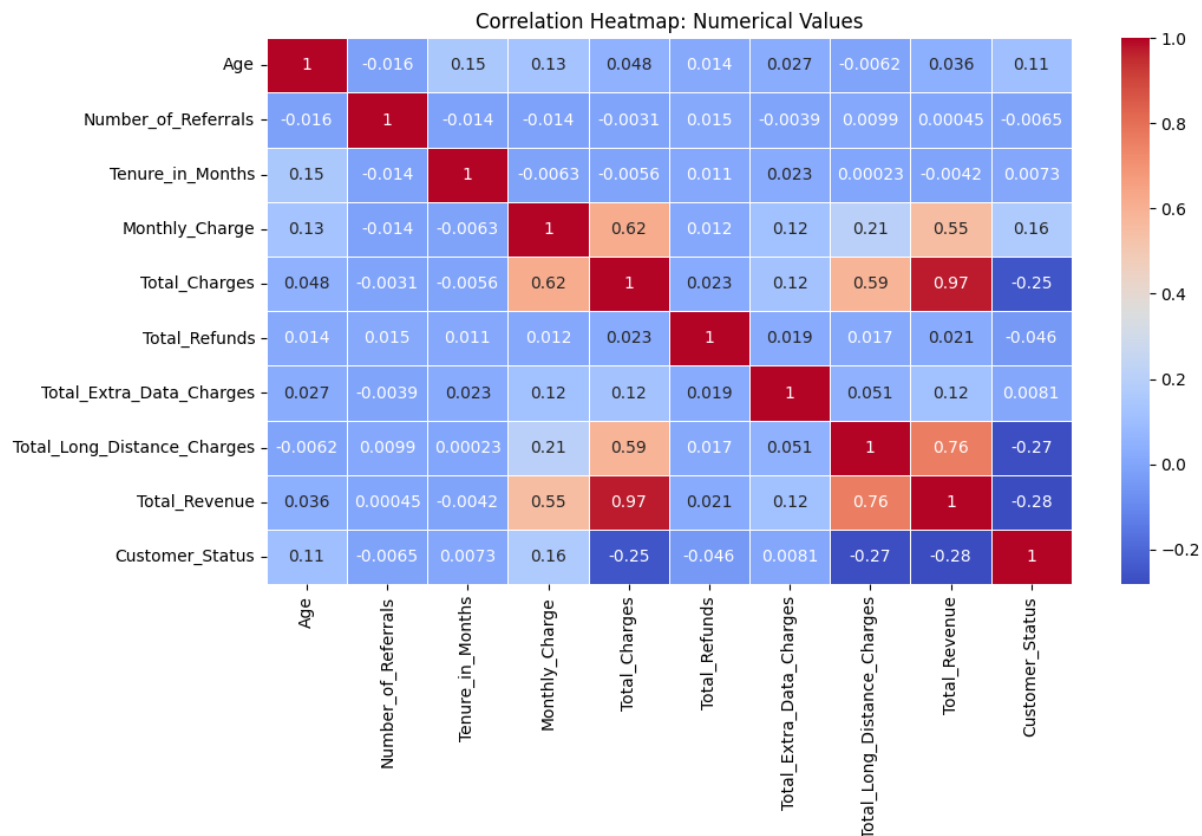


Figure 4: *Pearson correlation heatmap — numerical features and binary Customer Status. Strong multicollinearity is visible between Total_Charges, Total_Revenue, and Monthly_Charge.*

A few patterns stand out from the heatmap. `Total_Charges` and `Total_Revenue` are nearly perfectly correlated (0.97), which is expected since total revenue is derived in large part from charges. `Monthly_Charge` correlates with `Total_Charges` (0.62) and with `Total_Revenue` (0.55), reflecting the cumulative nature of billing. `Total_Long_Distance_Charges` shows a reasonably strong positive relationship with `Total_Revenue` (0.76).

Against the target `Customer_Status`, correlations are intentionally weak across all numerical features — the strongest is `Monthly_Charge` at 0.16. This confirms that no single numerical feature dominates the churn signal on its own, which is exactly why tree-based ensemble models that can capture non-linear interactions across all features were chosen over simpler linear classifiers. The multicollinearity between revenue-related columns was noted but not treated as a problem since XGBoost is robust to correlated inputs and no regularisation penalty is being applied at the feature level.

7.3 Step 3 — Data Preprocessing

With the EDA complete and no records removed, the preprocessing pipeline was assembled. The focus here is purely on encoding and structuring features correctly for the classifier —

null handling was already done in the SQL layer.

Listing 10: *Feature selection and target variable creation*

```

1 # Separate features (X) and target variable (y)
2 # Drop columns that would cause target leakage
3 columns_to_drop = [
4     "Churn_Category",
5     "Churn_Reason",
6     "Customer_ID",
7     "Customer_Status",
8 ]
9 X = df_churn.drop(columns=columns_to_drop, errors="ignore")
10
11 # Map target: Stayed = 0, Churned = 1
12 y = df_churn["Customer_Status"].map({"Stayed": 0, "Churned": 1})

```

Listing 11: *Stratified train-test split (80/20)*

```

1 from sklearn.model_selection import train_test_split
2
3 X_train, X_test, y_train, y_test = train_test_split(
4     X, y,
5     test_size=0.2,
6     random_state=42,
7     stratify=y      # preserves churn class proportion in both sets
8 )
9
10 print(f"Training set shape: {X_train.shape}") # (4805, 28)
11 print(f"Testing set shape: {X_test.shape}")   # (1202, 28)

```

Listing 12: *Categorical encoding via ColumnTransformer pipeline*

```

1 from sklearn.compose import ColumnTransformer
2 from sklearn.pipeline import Pipeline
3 from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder
4
5 # Nominal columns: no inherent order -> OneHotEncoder
6 nominal_cols = [
7     "Gender", "Married", "State", "Value_Deal",
8     "Phone_Service", "Multiple_Lines", "Internet_Service",
9     "Internet_Type", "Online_Security", "Online_Backup",
10    "Device_Protection_Plan", "Premium_Support",
11    "Streaming_TV", "Streaming_Movies", "Streaming_Music",
12    "Unlimited_Data", "Paperless_Billing", "Payment_Method",
13 ]
14

```

```

15 # Ordinal column: natural order -> OrdinalEncoder
16 ordinal_cols = ["Contract"]
17 contract_order = [["Month-to-Month", "One Year", "Two Year"]]
18
19 # Build the preprocessing pipeline
20 preprocessor = ColumnTransformer(
21     transformers=[
22         ("ord", OrdinalEncoder(categories=contract_order), ordinal_cols)
23         ,
24         ("nom", OneHotEncoder(handle_unknown="ignore",
25                                 sparse_output=False), nominal_cols),
26     ],
27     remainder="passthrough", # numerical columns pass through
28                               unchanged
29 )
30
31 # Output clean Pandas DataFrames instead of NumPy arrays
32 preprocessor.set_output(transform="pandas")
33
34 # Fit on training data only; transform both sets
35 X_train_processed = preprocessor.fit_transform(X_train)
36 X_test_processed = preprocessor.transform(X_test)

```

7.4 Step 4 — Model Training & Comparison

Three gradient-boosted and ensemble tree classifiers were trained and evaluated head-to-head. All used 150 estimators with class imbalance handling built in.

Listing 13: *Multi-model training and evaluation*

```

1 from xgboost import XGBClassifier
2 from lightgbm import LGBMClassifier
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.metrics import (
5     accuracy_score, f1_score, precision_score, recall_score
6 )
7
8 models = {
9     "Random Forest": RandomForestClassifier(
10         n_estimators=150,
11         max_depth=10,
12         random_state=42,
13         class_weight="balanced", # handles class imbalance
14     ),
15     "XGBoost": XGBClassifier(

```



```

16     n_estimators=150,
17     learning_rate=0.08,
18     max_depth=5,
19     random_state=42,
20     scale_pos_weight=2,           # weights minority class
21     eval_metric="logloss",
22 ),
23 "LightGBM": LGBMClassifier(
24     n_estimators=150,
25     learning_rate=0.08,
26     max_depth=5,
27     random_state=42,
28     is_unbalance=True,           # built-in imbalance handling
29 ),
30 }
31
32 print("--- MODEL PERFORMANCE EVALUATION ---\n")
33 for name, model in models.items():
34     model.fit(X_train_processed, y_train)
35     y_pred = model.predict(X_test_processed)
36     print(f"{'='*16} {name} {'='*16}")
37     print(f"Accuracy   : {accuracy_score(y_test, y_pred):.4f}")
38     print(f"Precision   : {precision_score(y_test, y_pred):.4f}")
39     print(f"Recall      : {recall_score(y_test, y_pred):.4f}")
40     print(f"F1-Score   : {f1_score(y_test, y_pred):.4f}\n")

```

7.4.1 Model Comparison Results

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	0.8378	0.7123	0.7349	0.7234
XGBoost	0.8386	0.7131	0.7378	0.7252
LightGBM	0.8228	0.6642	0.7810	0.7179

7.5 Step 5 — Model Selection Rationale

Why the Baseline XGBoost — Not the Tuned Version?

Hyperparameter tuning was explored and did push overall accuracy to **86.00%**. However, the tuned model's churn-class Recall dropped to **65.00%**, compared to **73.78%** for the baseline.

In customer retention, **Recall matters more than Accuracy**. Every missed

churner (false negative) is a lost customer the business never got the chance to retain. The baseline XGBoost was deliberately chosen for deployment because catching roughly **10% more actual churners** more than compensates for the 2% accuracy trade-off. False negatives are the expensive error here.

7.6 Step 6 — Model Serialisation

Listing 14: *Saving the preprocessor and model artefacts*

```
1 import joblib
2
3 # Select baseline XGBoost as the final production model
4 final_model = models["XGBoost"]
5
6 # Save the fitted preprocessing pipeline
7 joblib.dump(preprocessor, "churn_preprocessor.joblib")
8 print("Saved: churn_preprocessor.joblib")
9
10 # Save the trained model
11 joblib.dump(final_model, "baseline_xgb_model.joblib")
12 print("Saved: baseline_xgb_model.joblib")
```

7.7 Step 7 — Prediction on New Joiners

Listing 15: *Inference pipeline on new joiner dataset*

```
1 import joblib
2 import pandas as pd
3
4 # 1. Load serialised production pipeline components
5 loaded_preprocessor = joblib.load("churn_preprocessor.joblib")
6 loaded_model = joblib.load("baseline_xgb_model.joblib")
7
8 # 2. Prepare features (drop non-feature columns)
9 columns_to_drop = [
10     "Customer_ID", "Churn_Category",
11     "Churn_Reason", "Customer_Status",
12 ]
13 X_new = df_join.drop(columns=columns_to_drop, errors="ignore")
14
15 # 3. Transform using the fitted preprocessor
16 X_new_processed = loaded_preprocessor.transform(X_new)
17
```

```
18 # 4. Generate predictions and churn probabilities
19 df_join["Churn_Prediction"] = loaded_model.predict(X_new_processed)
20 df_join["Churn_Probability"] = (
21     loaded_model.predict_proba(X_new_processed)[: , 1]
22 )
23
24 # 5. Filter and sort high-risk customers
25 high_risk_customers = (
26     df_join[df_join["Churn_Prediction"] == 1]
27     .sort_values("Churn_Probability", ascending=False)
28 )
29
30 print(f"Found {len(high_risk_customers)} customers at risk.")
31 # Output: Found 388 customers at risk of churning.
32
33 # 6. Export to CSV for Power BI dashboard integration
34 high_risk_customers.to_csv("high_risk_churn_list.csv", index=False)
```

The top predicted churners carry churn probabilities exceeding 0.999, indicating extremely high model confidence for this subset. The full list of 388 at-risk customers was fed into both the Power BI Prediction dashboard and the ChurnRadar Streamlit application.

8 ChurnRadar — Streamlit Dashboard

ChurnRadar is a fully interactive, browser-based ML prediction dashboard built with Streamlit. It allows non-technical business users to upload the model artefacts and a customer CSV file and immediately see who is at risk — no Python environment, no notebooks, and no command-line work required beyond launching the app.

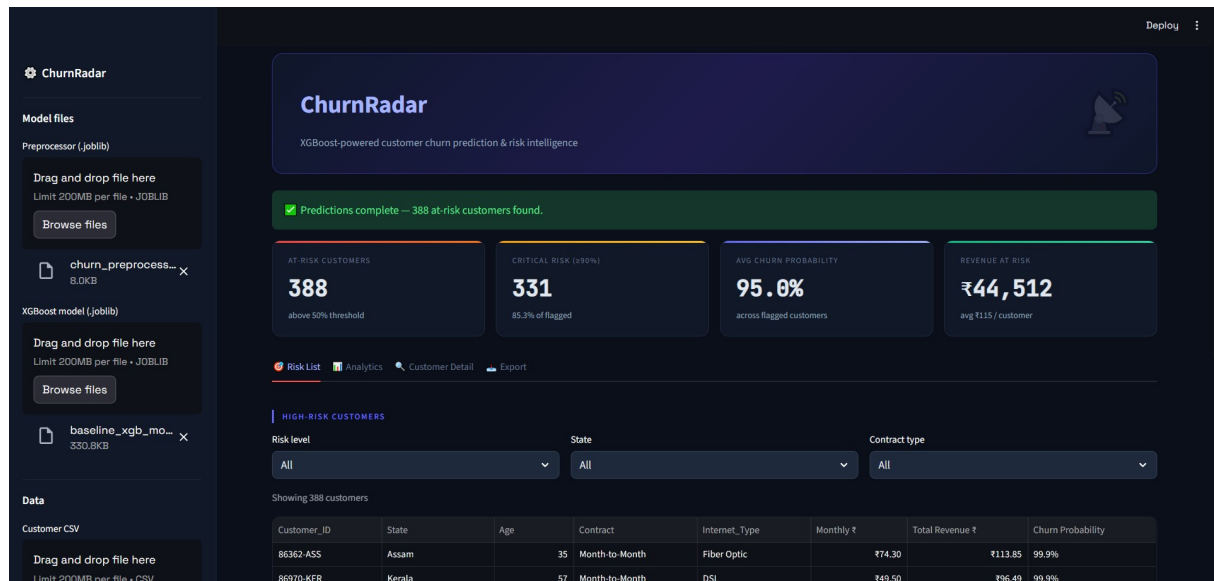


Figure 5: *ChurnRadar is a fully interactive, browser-based ML prediction dashboard built with Streamlit*

8.1 How It Works

Users upload three files via the sidebar: `churn_preprocessor.joblib`, `baseline_xgb_model.joblib`, and a customer CSV. Once all three are present, predictions run automatically through a `@st.cache_resource` pipeline. An adjustable probability threshold slider (default 50%) controls which customers appear in the dashboard.

8.2 Dashboard Tabs

1. **Risk List** — Filterable table of at-risk customers by risk level (Critical $\geq 90\%$ / High 75–90% / Medium $< 75\%$), State, and Contract type.
2. **Analytics** — Churn probability distribution histogram, top-10 states by at-risk count, contract and internet type breakdowns, Monthly Charge vs. Churn Probability scatter (bubble-sized by tenure).
3. **Customer Detail** — Individual customer profile lookup with full service subscription grid ($/\times$ icons) and financial metrics.
4. **Export** — Column-selector and one-click CSV download of the filtered at-risk list.

8.3 Risk Classification

Badge	Probability Range	Action Priority
Critical	$\geq 90\%$	Immediate outreach
High	75%–89%	Priority queue
Medium	50%–74%	Monitor and engage

9 Key Findings & Business Insights

9.1 Top Churn Risk Factors

1. **Contract Type (Month-to-Month)** — 46.5% churn rate; the single most powerful structural predictor in the dataset.
2. **Competitor Offers** — 761 customers explicitly cited competitors as their reason for leaving. This is both the largest churn category and the clearest call for competitive repositioning.
3. **Fiber Optic Internet** — 57.9% churn rate, pointing to reliability or pricing issues specific to this service tier.
4. **Mailed Check Payment** — 37.8% churn rate, correlating with lower digital engagement and account stickiness.
5. **Age Above 50** — 31.0% churn rate; the highest of any age segment, signalling a value-perception gap for older customers.
6. **Geographic Hotspots** — Jammu & Kashmir (57.2%), Assam (38.1%), and Jharkhand (34.5%) need dedicated attention.
7. **Low Service Bundling** — Customers without add-on services like Online Security and Premium Support churn at rates above 80% in those service non-subscription segments.

9.2 Retention Strategy Recommendations

Actionable Recommendations for the Business

- **Contract Upgrade Incentives:** Target Month-to-Month customers with loyalty discounts or exclusive benefits for switching to annual or two-year plans. Even a 10–15% conversion rate on this group would substantially reduce churn volume.
- **Competitive Counter-Offer Programme:** Build a rapid-response retention script specifically for customers who express intent to switch to competitors. Given 761 customers already left for this reason, proactive identification and outreach is essential.
- **Fiber Optic Quality Review:** The 57.9% churn rate in this segment is structurally abnormal. Investigate service reliability, pricing benchmarks against competitors, and support experience for Fiber customers specifically.
- **Service Bundling Campaigns:** Customers subscribing to Online Security, Premium Support, and Online Backup churn at dramatically lower rates. A targeted "bundle and save" promotion for single-service customers could meaningfully improve retention.
- **Geographic Focus:** Deploy customer success representatives or regional campaigns in Jammu & Kashmir, Assam, and Jharkhand for historical churn hotspots. For forward-looking action, prioritise Uttar Pradesh (45), Maharashtra (41), and Tamil Nadu (36) from the predicted churning list.
- **Older Customer Engagement:** Design dedicated communication, support, and value-demonstration programmes for customers aged 50+. This group has both the highest churn rate and a large customer count — small retention gains here translate to significant revenue impact.
- **Act on the 388 Predicted Churners Now:** The model has flagged 388 newly joined customers representing **44,512 in revenue at risk**. These customers should enter a retention workflow immediately — before churn is realised — using the ChurnRadar dashboard to prioritise outreach by probability score.

10 How to Run This Project

10.1 Prerequisites

- Microsoft SQL Server (Express or higher) with ODBC Driver 17 installed

- Power BI Desktop
- Python 3.10 or later with the following packages:

Listing 16: *Python package installation*

```
pip install pandas numpy scikit-learn xgboost lightgbm joblib \
    sqlalchemy pyodbc streamlit matplotlib seaborn
```

10.2 Step-by-Step Setup

1. **SQL Setup** — Run the six T-SQL scripts in order (01 through 06) against your SQL Server instance. Update the file path inside the `BULK INSERT` statement to point to your local `Customer_Data.csv`.
2. **ML Training** — Open `Machine Learning Predictions/Churn_Predictions.ipynb` and run all cells. The notebook connects to SQL Server, trains the XGBoost model, and exports three files:
 - `churn_preprocessor.joblib`
 - `baseline_xgb_model.joblib`
 - `high_risk_churn_list.csv`
3. **Power BI** — Open `Churn Analysis.pbix` in Power BI Desktop. Refresh the SQL Server connection pointing to `db_Churn`. For the Prediction page, connect to the exported `high_risk_churn_list.csv`.
4. **Streamlit Dashboard** — Launch ChurnRadar from the terminal:

```
streamlit run "Streamlit App/churn_prediction.py"
```

In the sidebar, upload the two `.joblib` files and your customer CSV. The dashboard populates automatically.

11 Conclusion

This project demonstrates what a genuine end-to-end data science workflow looks like when built with production use in mind rather than exploratory notebooks alone. The pipeline flows cleanly from raw CSV ingestion through SQL Server, into business intelligence via Power BI, into a properly engineered scikit-learn machine learning pipeline, and finally out through a live Streamlit dashboard that any business user can operate.

The numbers are concrete: a 27% historical churn rate, 388 new customers flagged as at risk, and 44,512 in revenue sitting in the at-risk column. The analysis around contract type, competitor pressure, Fiber Optic service quality, age-related value gaps, and geographic concentration gives the business clear and prioritised places to direct retention effort right now.

The broader architecture — staging and production table separation in SQL, view-based data contracts between layers, a serialised sklearn preprocessing pipeline, and a Streamlit app that loads the pipeline at runtime — is a pattern that holds up at scale. Swap the dataset for banking transactions, e-commerce purchase histories, or SaaS usage logs, and the same framework applies. The domain changes; the pipeline does not.

Shehraz Sarwar

Aspiring Data Scientist | IBM Certified Data Analyst
Ex Data Science Intern @10Pearls | Section Leader Stanford CIP '25

 [LinkedIn](#)  [GitHub](#)